

```
// AISC PRACTICAL EXAM MASTER CODES - READABLE MACRO PRINT
// 24CE1045 Aaryan Choube | Batch A/A1 | 7.0 pt Consolas
```

```
=====
[AISC-02] WaterJugSolver.java (Exp 2)
Complexity: Time:  $O(C1 * C2)$  | Space:  $O(C1 * C2)$  | Concept: State
space BFS graph traversal for measuring 2 gallons
-----
```

```
import java.util.*;

public class WaterJugSolver {
    static class State {
        int x, y;
        List<String> path;

        State(int x, int y, List<String> path) {
            this.x = x;
            this.y = y;
            this.path = new ArrayList<>(path);
            this.path.add("(" + x + ", " + y + ")");
        }
    }

    public static void solveBFS(int jug1, int jug2, int target) {
        System.out.println("\nSolving Water Jug Problem using BFS (Shortest
Path):");
        System.out.println("Jug 1 Capacity: " + jug1 + "L | Jug 2 Capacity: "
+ jug2 + "L | Target: " + target + "L");

        if (target > Math.max(jug1, jug2) || target % gcd(jug1, jug2) != 0) {
            System.out.println("Target cannot be measured according to Bezout's
identity.");
            return;
        }

        Queue<State> queue = new LinkedList<>();
        boolean[][] visited = new boolean[jug1 + 1][jug2 + 1];

        queue.add(new State(0, 0, new ArrayList<>()));
        visited[0][0] = true;

        while (!queue.isEmpty()) {
            State current = queue.poll();

            if (current.x == target || current.y == target) {
                System.out.println("\nGoal State Reached!");
                System.out.println("Steps to solution (" + (current.path.size() -
1) + " operations):");
                for (int i = 0; i < current.path.size(); i++) {
                    System.out.println("Step " + i + ": " + current.path.get(i));
                }
                return;
            }

            List<int[]> nextStates = new ArrayList<>();
            nextStates.add(new int[]{jug1, current.y}); // Fill Jug 1
            nextStates.add(new int[]{current.x, jug2}); // Fill Jug 2
            nextStates.add(new int[]{0, current.y}); // Empty Jug 1
            nextStates.add(new int[]{current.x, 0}); // Empty Jug 2

            int pour1to2 = Math.min(current.x, jug2 - current.y);
            nextStates.add(new int[]{current.x - pour1to2, current.y +
pour1to2});

            int pour2to1 = Math.min(current.y, jug1 - current.x);
            nextStates.add(new int[]{current.x + pour2to1, current.y -
pour2to1});

            for (int[] next : nextStates) {
                int nx = next[0], ny = next[1];
                if (!visited[nx][ny]) {
                    visited[nx][ny] = true;
                    queue.add(new State(nx, ny, current.path));
                }
            }
        }
        System.out.println("No solution exists.");
    }

    private static int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("=====");
        System.out.println("    Water Jug Problem Solver (AISC Exp 2)");
        System.out.println("=====");
        System.out.println("1. Use default: Jug1 = 4L, Jug2 = 3L, Target =
2L");
        System.out.println("2. Enter custom values");
        System.out.print("Enter choice (1/2): ");
    }
}
```

```
int choice = 1;
if (sc.hasNextInt()) {
    choice = sc.nextInt();
}

int j1 = 4, j2 = 3, target = 2;
if (choice == 2) {
    System.out.print("Enter capacity of Jug 1: ");
    j1 = sc.nextInt();
    System.out.print("Enter capacity of Jug 2: ");
    j2 = sc.nextInt();
    System.out.print("Enter target amount: ");
    target = sc.nextInt();
}

solveBFS(j1, j2, target);
System.out.println("=====");
}
```

```
=====
[AISC-03] TicTacToeMinimax.java (Exp 3)
Complexity: Time:  $O(b^d)$  | Space:  $O(d)$  | Concept: Adversarial game-
tree search for optimal move selection
-----
```

```
import java.util.Scanner;

public class TicTacToeMinimax {
    static final char HUMAN = 'X';
    static final char AI = 'O';
    static final char EMPTY = ' ';

    public static void printBoard(char[][] board) {
        System.out.println("");
        for (int i = 0; i < 3; i++) {
            System.out.println(" " + board[i][0] + " | " + board[i][1] + " | "
+ board[i][2]);
            if (i < 2) System.out.println("---|---|---");
        }
        System.out.println("");
    }

    public static boolean isMovesLeft(char[][] board) {
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                if (board[i][j] == EMPTY) return true;
        return false;
    }

    public static int evaluate(char[][] b) {
        for (int row = 0; row < 3; row++) {
            if (b[row][0] == b[row][1] && b[row][1] == b[row][2]) {
                if (b[row][0] == AI) return +10;
                else if (b[row][0] == HUMAN) return -10;
            }
        }
        for (int col = 0; col < 3; col++) {
            if (b[0][col] == b[1][col] && b[1][col] == b[2][col]) {
                if (b[0][col] == AI) return +10;
                else if (b[0][col] == HUMAN) return -10;
            }
        }
        if (b[0][0] == b[1][1] && b[1][1] == b[2][2]) {
            if (b[0][0] == AI) return +10;
            else if (b[0][0] == HUMAN) return -10;
        }
        if (b[0][2] == b[1][1] && b[1][1] == b[2][0]) {
            if (b[0][2] == AI) return +10;
            else if (b[0][2] == HUMAN) return -10;
        }
        return 0;
    }

    public static int minimax(char[][] board, int depth, boolean isMax) {
        int score = evaluate(board);
        if (score == 10) return score - depth;
        if (score == -10) return score + depth;
        if (!isMovesLeft(board)) return 0;

        if (isMax) {
            int best = -1000;
            for (int i = 0; i < 3; i++) {
                for (int j = 0; j < 3; j++) {
                    if (board[i][j] == EMPTY) {
                        board[i][j] = AI;
                        best = Math.max(best, minimax(board, depth + 1, false));
                        board[i][j] = EMPTY;
                    }
                }
            }
        }
        return best;
    }
}
```

```

    } else {
        int best = 1000;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[i][j] == EMPTY) {
                    board[i][j] = HUMAN;
                    best = Math.min(best, minimax(board, depth + 1, true));
                    board[i][j] = EMPTY;
                }
            }
        }
        return best;
    }
}

public static int[] findBestMove(char[][] board) {
    int bestVal = -1000;
    int[] bestMove = {-1, -1};
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[i][j] == EMPTY) {
                board[i][j] = AI;
                int moveVal = minimax(board, 0, false);
                board[i][j] = EMPTY;
                if (moveVal > bestVal) {
                    bestMove[0] = i;
                    bestMove[1] = j;
                    bestVal = moveVal;
                }
            }
        }
    }
    return bestMove;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("=====");
    System.out.println("  Tic-Tac-Toe Minimax Agent (AISC Exp 3)");
    System.out.println("=====");
    System.out.println("You are 'X' and AI is 'O'. Position numbers 1 to 9:");
    System.out.println(" 1 | 2 | 3 ");
    System.out.println("---|---|---");
    System.out.println(" 4 | 5 | 6 ");
    System.out.println("---|---|---");
    System.out.println(" 7 | 8 | 9 ");

    char[][] board = {
        {' ', ' ', ' '},
        {' ', ' ', ' '},
        {' ', ' ', ' '}
    };

    System.out.println("\nAI calculating optimal response if Human plays center (pos 5):");
    board[1][1] = HUMAN;
    printBoard(board);

    int[] aiMove = findBestMove(board);
    System.out.println("Minimax AI selects row " + aiMove[0] + ", col " + aiMove[1] + " (Optimal Corner response)");
    board[aiMove[0]][aiMove[1]] = AI;
    printBoard(board);

    System.out.println("Strategy: Minimax Decision Tree (Adversarial Search)");
    System.out.println("Result: AI is theoretically unbeatable (always wins or draws).");
    System.out.println("=====");
}

```

[AISC-05] AStarSearch.java (Exp 5)
Complexity: Time: $O(b^d)$ | Space: $O(b^d)$ | Concept: Informed heuristic graph search evaluating $f(n) = g(n) + h(n)$

```
import java.util.*;
```

```

public class AStarSearch {
    static class Node implements Comparable<Node> {
        int id;
        int g;
        int h;
        int f;
        Node parent;

        Node(int id, int g, int h, Node parent) {
            this.id = id;
            this.g = g;
            this.h = h;

```

```

        this.f = g + h;
        this.parent = parent;
    }

    public int compareTo(Node o) {
        return Integer.compare(this.f, o.f);
    }
}

static class Edge {
    int target, weight;
    Edge(int target, int weight) {
        this.target = target;
        this.weight = weight;
    }
}

public static void runAStar(Map<Integer, List<Edge>> graph, int[]
heuristics, int start, int goal) {
    PriorityQueue<Node> openList = new PriorityQueue<>();
    Map<Integer, Integer> bestG = new HashMap<>();

    openList.add(new Node(start, 0, heuristics[start], null));
    bestG.put(start, 0);

    System.out.println("\n--- A* Search Node Expansion ---");
    System.out.println("Node\tg(n)\th(n)\tf(n)");

    while (!openList.isEmpty()) {
        Node current = openList.poll();
        System.out.printf("N%d\t%d\t%d\t%d\n", current.id, current.g,
current.h, current.f);

        if (current.id == goal) {
            System.out.println("\nGoal Node " + goal + " reached with optimal
cost: " + current.g);
            printPath(current);
            return;
        }

        for (Edge edge : graph.getOrDefault(current.id,
Collections.emptyList())) {
            int newG = current.g + edge.weight;
            int neighbor = edge.target;

            if (!bestG.containsKey(neighbor) || newG < bestG.get(neighbor)) {
                bestG.put(neighbor, newG);
                openList.add(new Node(neighbor, newG, heuristics[neighbor],
current));
            }
        }
        System.out.println("Goal node is unreachable.");
    }

    private static void printPath(Node goalNode) {
        List<Integer> path = new ArrayList<>();
        Node curr = goalNode;
        while (curr != null) {
            path.add(curr.id);
            curr = curr.parent;
        }
        Collections.reverse(path);
        System.out.print("Optimal Path: ");
        for (int i = 0; i < path.size(); i++) {
            System.out.print("N" + path.get(i) + (i < path.size() - 1 ? " -> "
: ""));
        }
        System.out.println();
    }

    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println("      A* Search Algorithm (AISC Exp 4)");
        System.out.println("=====");

        int[] heuristics = {10, 8, 5, 7, 3, 0};
        Map<Integer, List<Edge>> graph = new HashMap<>();
        for (int i = 0; i < 6; i++) graph.put(i, new ArrayList<>());

        graph.get(0).add(new Edge(1, 2));
        graph.get(0).add(new Edge(2, 4));
        graph.get(1).add(new Edge(3, 7));
        graph.get(1).add(new Edge(4, 3));
        graph.get(2).add(new Edge(4, 1));
        graph.get(3).add(new Edge(5, 5));
        graph.get(4).add(new Edge(5, 4));

        System.out.println("Graph configured with Start = N0, Goal = N5");
        System.out.println("Heuristics h(n): N0=10, N1=8, N2=5, N3=7, N4=3,
N5=0");
    }
}

```

```

runAStar(graph, heuristics, 0, 5);
System.out.println("Evaluation Function: f(n) = g(n) + h(n)");
System.out.println("Optimality: A* is optimal if heuristic h(n) is
admissible (never overestimates).");
System.out.println("=====");
}
}

=====
[AISC-06] FuzzyOperations.java (Exp 6)
Complexity: Time: O(n) | Space: O(n) | Concept: Fuzzy set theory:
Union (max), Intersection (min), Complement (1-mu)
-----

import java.util.Scanner;

public class FuzzyOperations {
    public static void printSet(String name, double[] set) {
        System.out.print(name + " = { ");
        for (int i = 0; i < set.length; i++) {
            System.out.printf("%.2f/x%d", set[i], (i + 1));
            if (i < set.length - 1) System.out.print(", ");
        }
        System.out.println(" }");
    }

    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println("      Fuzzy Set Operations (AISC Exp 8)");
        System.out.println("=====");

        double[] A = {0.2, 0.5, 0.8, 1.0};
        double[] B = {0.6, 0.4, 0.7, 0.1};

        printSet("Fuzzy Set A", A);
        printSet("Fuzzy Set B", B);

        int n = A.length;
        double[] union = new double[n];
        double[] intersection = new double[n];
        double[] compA = new double[n];
        double[] compB = new double[n];
        double[] diffAB = new double[n];

        for (int i = 0; i < n; i++) {
            union[i] = Math.max(A[i], B[i]);
            intersection[i] = Math.min(A[i], B[i]);
            compA[i] = 1.0 - A[i];
            compB[i] = 1.0 - B[i];
            diffAB[i] = Math.min(A[i], 1.0 - B[i]);
        }

        System.out.println("\n--- Standard Fuzzy Operations ---");
        printSet("Union (A U B)      [max]", union);
        printSet("Intersection (A n B) [min]", intersection);
        printSet("Complement (A')     [1 - A]", compA);
        printSet("Difference (A - B)   [A n B']", diffAB);

        System.out.println("\n--- Cartesian Product (A x B) ---");
        double[][] cartesian = new double[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                cartesian[i][j] = Math.min(A[i], B[j]);
                System.out.printf("%.2f\t", cartesian[i][j]);
            }
            System.out.println();
        }

        System.out.println("\n--- Max-Min Composition Demo ---");
        double[][] R = {{0.3, 0.7}, {0.8, 0.4}};
        double[][] S = {{0.5, 0.9}, {0.6, 0.2}};
        double[][] T = new double[2][2];

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                double maxVal = 0.0;
                for (int k = 0; k < 2; k++) {
                    maxVal = Math.max(maxVal, Math.min(R[i][k], S[k][j]));
                }
                T[i][j] = maxVal;
            }
        }

        System.out.println("Composition Matrix T = R o S:");
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                System.out.printf("%.2f\t", T[i][j]);
            }
            System.out.println();
        }
        System.out.println("=====");
    }
}

```

```

=====
[AISC-10] Perceptron.java (Exp 10)
Complexity: Time: O(Epochs * N) | Space: O(Features) | Concept:
Single-layer artificial neural network binary classification
-----

import java.util.Scanner;

public class Perceptron {
    public static void trainPerceptron(int[][] inputs, int[] targets,
String gateName) {
        double[] weights = {0.0, 0.0};
        double bias = 0.0;
        double lr = 0.1;
        int epochs = 10;

        System.out.println("\n--- Training Perceptron for " + gateName + "
Gate ---");
        System.out.println("Learning Rate: " + lr);

        for (int epoch = 1; epoch <= epochs; epoch++) {
            int totalError = 0;
            for (int i = 0; i < inputs.length; i++) {
                double net = inputs[i][0] * weights[0] + inputs[i][1] *
weights[1] + bias;
                int output = (net >= 0) ? 1 : 0;
                int error = targets[i] - output;
                totalError += Math.abs(error);

                weights[0] += lr * error * inputs[i][0];
                weights[1] += lr * error * inputs[i][1];
                bias += lr * error;
            }

            if (totalError == 0) {
                System.out.println("Converged at Epoch " + epoch + "!");
                break;
            }

            System.out.printf("Final Weights: w1 = %.2f, w2 = %.2f, bias =
%.2f\n", weights[0], weights[1], bias);
            System.out.println("Verification Table:");
            System.out.println("x1\tx2\tTarget\tPredicted\tStatus");
            for (int i = 0; i < inputs.length; i++) {
                double net = inputs[i][0] * weights[0] + inputs[i][1] * weights[1]
+ bias;
                int pred = (net >= 0) ? 1 : 0;
                System.out.println(inputs[i][0] + "\t" + inputs[i][1] + "\t" +
targets[i] + "\t" + pred + "\t\t" + (pred == targets[i] ? "MATCH" :
"FAIL"));
            }
        }

        public static void main(String[] args) {
            System.out.println("=====");
            System.out.println("      Single-Layer Perceptron (AISC Exp 6)");
            System.out.println("=====");

            int[][] inputs = {
                {0, 0},
                {0, 1},
                {1, 0},
                {1, 1}
            };

            int[] andTargets = {0, 0, 0, 1};
            int[] orTargets = {0, 1, 1, 1};

            trainPerceptron(inputs, andTargets, "AND");
            trainPerceptron(inputs, orTargets, "OR");

            System.out.println("\nNote: SLP can solve linearly separable problems
(AND, OR).");
            System.out.println("XOR is non-linearly separable and requires Multi-
Layer Perceptron (MLP).");
            System.out.println("=====");
        }
    }
}

```